



**S. B. JAIN INSTITUTE OF TECHNOLOGY,
MANAGEMENT & RESEARCH, NAGPUR.**

Post Lab 02

Aim: Convert images between the RGB, HSV, YCrCb, and Lab colour spaces, analyzing how colour information is encoded in each.

Name of Student:

Roll No.:

Semester/Year: V/III

Academic Session: 2026-2027

Date of Performance:

Date of Submission:

Aim: Convert images between the RGB, HSV, YCrCb, and Lab colour spaces, analyzing how colour information is encoded in each.

OBJECTIVE/EXPECTED LEARNING OUTCOME:

The objectives and expected learning outcome of this practical are:

- To understand and apply color space conversion techniques in OpenCV by transforming images between RGB, HSV, YCrCb, and Lab formats.
- To analyze the differences in color representation across RGB, HSV, YCrCb, and Lab color spaces, emphasizing how hue, brightness, and chrominance are encoded.
- To visualize and interpret individual channels (e.g., Hue, Saturation, Lightness, Cr, Cb) to assess how different color spaces isolate color and intensity components.
- To evaluate the suitability of each color space for various image processing tasks such as enhancement, segmentation, and compression

THEORY:

What is a Color Model?

A color model is a mathematical way to describe and represent colors using numbers – typically in tuple like RGB (three values: Red, Green, Blue) or quadruple like CMYK (four values: Cyan, Magenta, Yellow, Black). These values define positions in a coordinate system that helps us encode color information digitally.

For a given color model, to interpret a tuple or quadruple as a color, we can define a set of rules and definitions used to accurately calibrate and generate colors, i.e. a mapping function. A color space identifies a specific combination of color models and mapping functions. Identifying the color space automatically identifies the associated color model. For example, Adobe RGB and sRGB are two different color spaces, both based on the RGB color model.

1. RGB (Red, Green, Blue) Color Space

- Description: The RGB model represents color through the combination of red, green, and blue light.
- Encoding: Each pixel contains three values (R, G, B), typically ranging from 0–255.
- Use Case: Ideal for display and basic image storage, but less effective for tasks that require separating brightness and color.
- Limitation: Not perceptually uniform — equal changes in R, G, or B don't correspond to equal changes in perceived color.

2. HSV (Hue, Saturation, Value) Color Space

- Description: HSV models color similarly to how humans perceive it. It separates the chromatic content (Hue and Saturation) from the luminance (Value).
- Encoding:
 - Hue (H): Represents the type of color (0–179 in OpenCV).
 - Saturation (S): Measures the vibrancy of the color (0–255).

- Value (V): Indicates brightness (0–255).
- Use Case: Excellent for image segmentation, object detection, and color-based filtering (e.g., detecting red objects regardless of lighting).
- Advantage: Decouples color from brightness, making it robust under varying lighting conditions.

3. YCrCb (Luminance-Chrominance) Color Space

- Description: Primarily used in video compression standards like JPEG and MPEG.
- Encoding:
 - Y: Luminance (brightness).
 - Cr: Difference between red and luminance.
 - Cb: Difference between blue and luminance.
- Use Case: Efficient in compression and transmission since the human eye is more sensitive to luminance than color differences.
- Advantage: Enables chroma subsampling (e.g., 4:2:0 format), reducing data without significant loss in visual quality.

4. Lab (CIELAB) Color Space

- Description: A perceptually uniform color space designed to approximate human vision.
- Encoding:
 - L*: Lightness (0–100).
 - a*: Green–Red axis.
 - b*: Blue–Yellow axis.
- Use Case: Ideal for color correction, enhancement, and perceptual comparisons.
- Advantage: Color differences in Lab closely match human visual perception; useful in industries like printing and textiles.

Gray Scale Image :

Grayscale image contains only single channel. Pixel intensities in this color space is represented by values ranging from 0 to 255. Thus, number of possibilities for one color represented by a pixel is 256.

Here are some common options for the conversion flag:

- **BGR to other color spaces:**
 - cv2.COLOR_BGR2RGB: Converts a BGR image to RGB.
 - cv2.COLOR_BGR2GRAY: Converts a BGR image to grayscale.
 - cv2.COLOR_BGR2HSV: Converts a BGR image to HSV (Hue, Saturation, Value).
 - cv2.COLOR_BGR2HLS: Converts a BGR image to HLS (Hue, Luminance, Saturation).
 - cv2.COLOR_BGR2LAB: Converts a BGR image to LAB color space.
- **RGB to other color spaces:**
 - cv2.COLOR_RGB2BGR: Converts an RGB image to BGR.
 - cv2.COLOR_RGB2GRAY: Converts an RGB image to grayscale.
 - cv2.COLOR_RGB2HSV: Converts an RGB image to HSV.
 - cv2.COLOR_RGB2HLS: Converts an RGB image to HLS.
 - cv2.COLOR_RGB2LAB: Converts an RGB image to LAB.
 - cv2.COLOR_RGB2YCrCb: Converts an RGB image to YCrCb.

- **Grayscale to other color spaces:**
 - cv2.COLOR_GRAY2BGR: Converts a grayscale image to BGR.
 - cv2.COLOR_GRAY2RGB: Converts a grayscale image to RGB.
- **Other conversions:**
 - Conversions between HSV, HLS, LAB, and YCrCb and other color spaces (e.g., cv2.COLOR_HSV2BGR, cv2.COLOR_LAB2RGB, etc.) also exist.

CODE:

Gray Color

```
import cv2
```

```
# Read image directly in grayscale mode
```

```
img = cv2.imread('color.png', cv2.IMREAD_GRAYSCALE)
```

- `cv2.imread()` is a function used to read an image.
- `'g4g.png'` is the filename of the image you want to load. Make sure the file is in the same directory or provide a full path.
- `cv2.IMREAD_GRAYSCALE` tells OpenCV to load the image in **grayscale mode** (1 channel only, instead of 3 in RGB).
- The result is stored in the variable `img`.

```
# Check if the image was loaded properly
```

```
if img is None:
```

```
    print("Error: Image not found or unable to load.")
```

```
else:
```

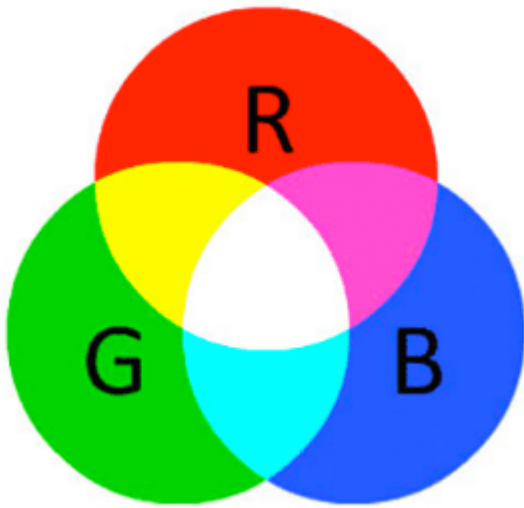
```
    # Show the grayscale image
```

```
    cv2.imshow('Grayscale Image', img)
```

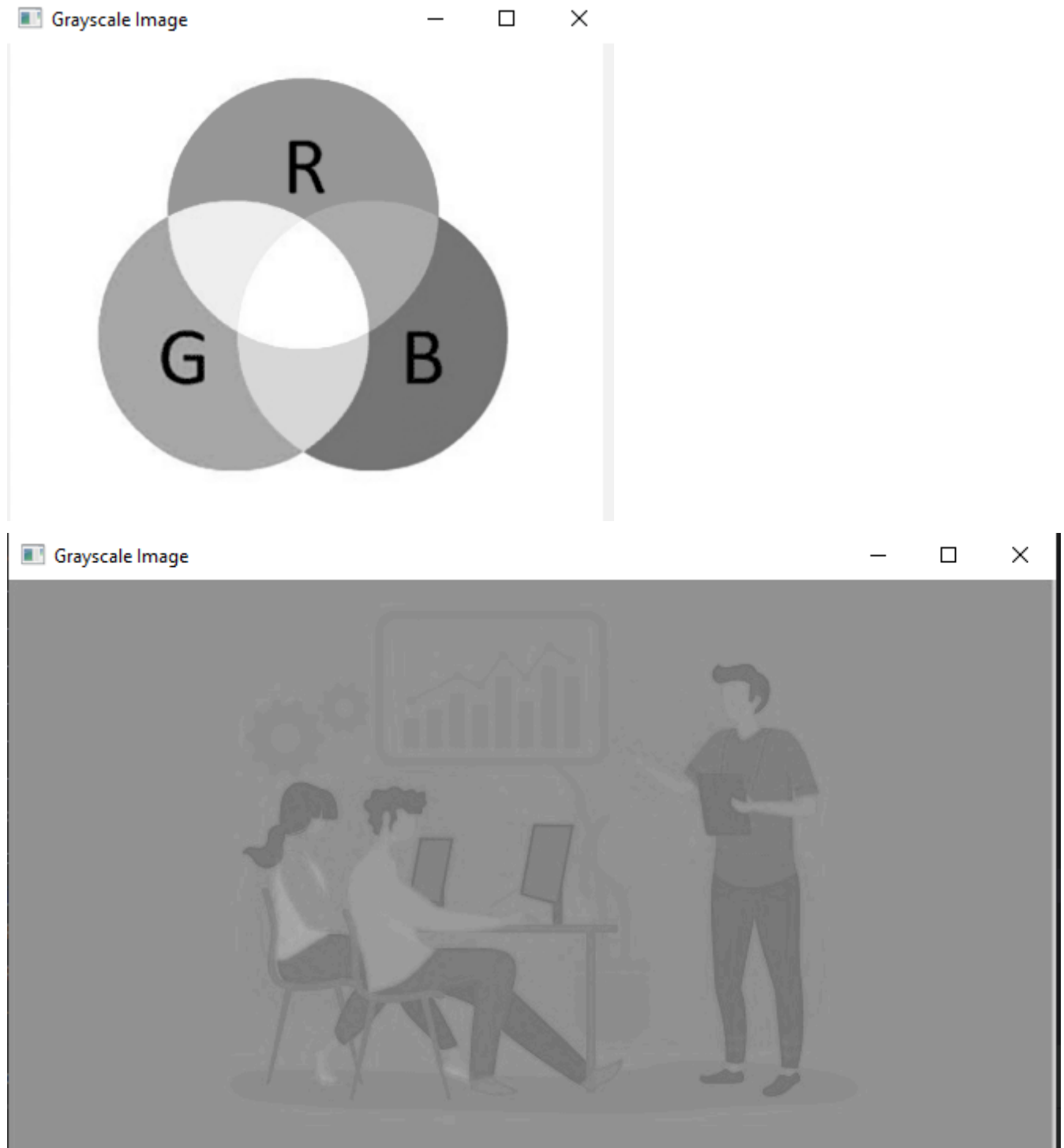
```
    cv2.waitKey(0)
```

```
    cv2.destroyAllWindows()
```

INPUT:-



OUTPUT:



CODE2:

```
import cv2
```

```
# Read the image
```

```
image = cv2.imread('cspace.png')
```

Department of Computer Science and Engineering, S.B.J.I.T.M.R., Nagpur

```
# Check if image is loaded properly
if image is None:
    print("Error: Image not found or unable to load.")
else:
    # Split the image into B, G, R channels
    B, G, R = cv2.split(image)

    # Show the original image and wait for a key press
    cv2.imshow("Original", image)
    cv2.waitKey(0)

    # Show blue channel
    cv2.imshow("Blue", B)
    cv2.waitKey(0)

    # Show green channel
    cv2.imshow("Green", G)
    cv2.waitKey(0)

    # Show red channel
    cv2.imshow("Red", R)
    cv2.waitKey(0)

cv2.destroyAllWindows()
```

INPUT:



OUTPUT: Note:- Four Different output will be shown one after another for above code.

YCrCb Color Space :

Department of Computer Science and Engineering, S.B.J.I.T.M.R., Nagpur

Y represents Luminance or Luma component, Cb and Cr are Chroma components. Cb represents the blue-difference (difference of blue component and Luma Component). Cr represents the red-difference (difference of red component and Luma Component).

CODE:

```
# Python program to read image
# as YCrCb color space

# Import cv2 module
import cv2

# Reads the image
img = cv2.imread('color2.png')

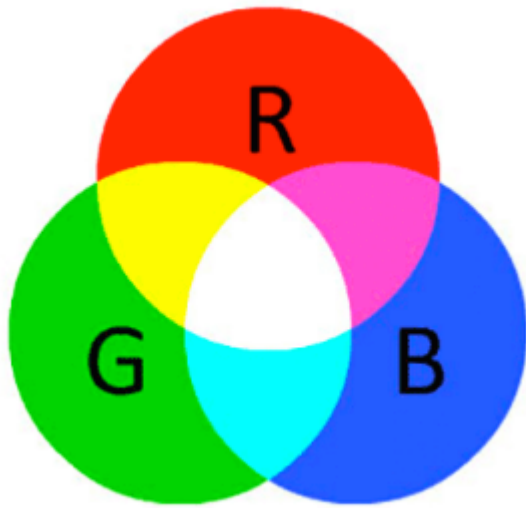
# Convert to YCrCb color space
img = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)
```

- Converts the image from **BGR to YCrCb** using `cv2.cvtColor()` .
- **YCrCb** is a color space that separates luminance (Y) from chrominance (Cr and Cb):
 - **Y**: Luma (brightness).
 - **Cr**: Difference between red and luma.
 - **Cb**: Difference between blue and luma.
- This format is commonly used in video compression and broadcasting.

```
# Shows the image
cv2.imshow('image', img)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



HSV color space :

H : Hue represents dominant wavelength. **S** : Saturation represents shades of color. **V** : Value represents Intensity.

CODE:-

```
# Python program to read image
# as HSV color space

# Importing cv2 module
import cv2

# Reads the image
img = cv2.imread('color2.png')

# Converts to HSV color space
img = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

- Converts the image from **BGR to HSV** color space.
- In HSV:
 - **H (Hue)**: Represents color type (0–179 in OpenCV).
 - **S (Saturation)**: Intensity or purity of the color (0–255).
 - **V (Value)**: Brightness or lightness (0–255).
- HSV is often used in color detection and filtering.

Shows the image

```
cv2.imshow('image', img)
```

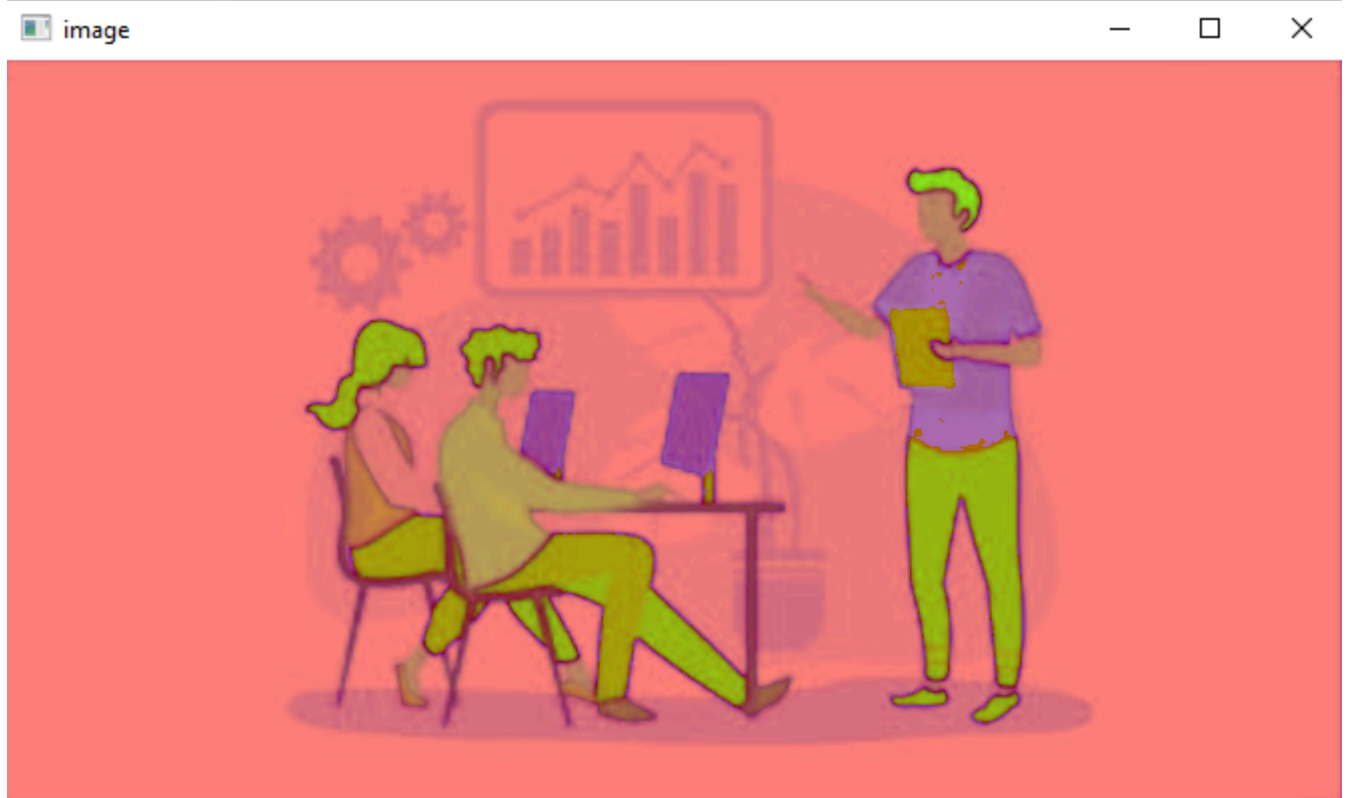
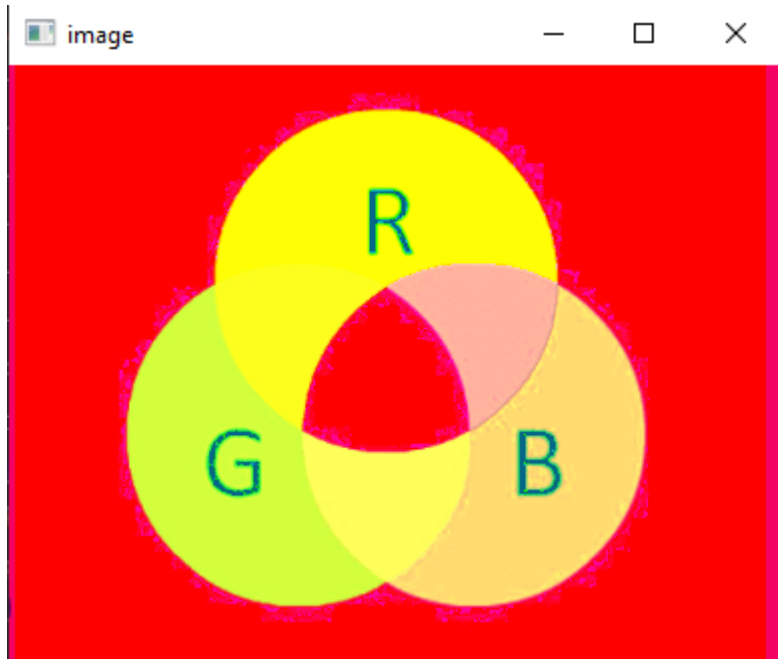
```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:



LAB color space :

Department of Computer Science and Engineering, S.B.J.I.T.M.R., Nagpur

L - Represents Lightness. **A** – Color component ranging from Green to Magenta. **B** – Color component ranging from Blue to Yellow.

CODE:

```
import cv2

# Read the image
image = cv2.imread('color2.png')

# Check if image was loaded successfully
if image is None:
    print("Error: Image not found or failed to load.")
    exit()

# Convert the image to LAB color space
lab_image = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)

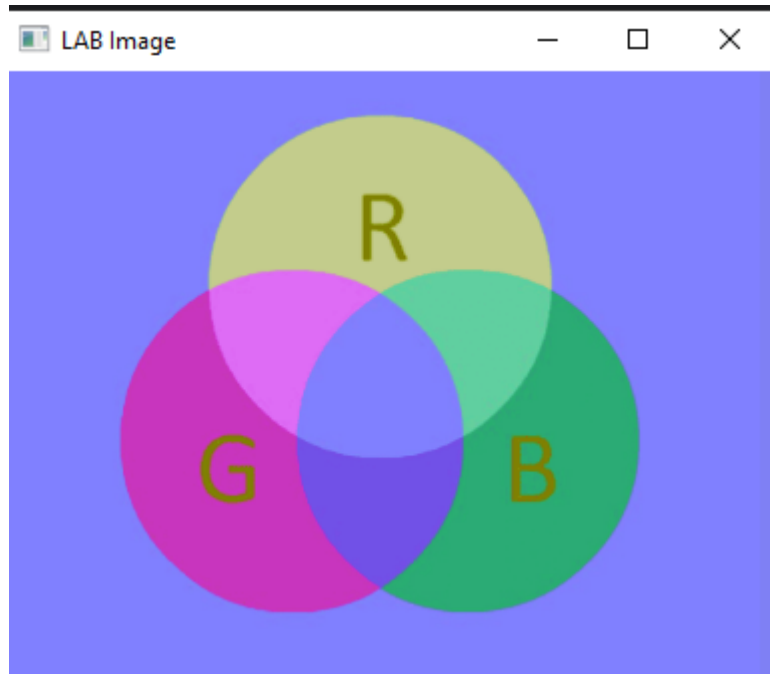
    • Converts the BGR image to the LAB color space using cv2.cvtColor() .
    • LAB stands for:
        • L: Lightness (0–100)
        • A: Green to Red component
        • B: Blue to Yellow component
    • LAB is perceptually uniform, meaning small changes in values represent similar changes in human perception—useful for color-based image processing.

# Display the LAB image
cv2.imshow('LAB Image', lab_image)

# Wait for a key press and close all OpenCV windows
cv2.waitKey(0)
cv2.destroyAllWindows()
```

INPUT: Same images as above

OUTPUT:



Note: All snapshot to be added with your name as caption for all program and try different input images for better understanding.

CONCLUSION:

DISCUSSION QUESTIONS:

1. How does the separation of chromatic and luminance components in HSV, YCrCb, and Lab help in image processing tasks like segmentation or enhancement compared to RGB?
2. In what scenarios would HSV be more suitable than RGB for image thresholding or object tracking?
3. Why is the Lab color space considered more perceptually uniform compared to RGB and HSV? How does this benefit color-based applications?
4. What challenges might arise when converting between color spaces, and how can data loss or distortion be minimized?
5. How do individual channels (e.g., Hue in HSV, Cr in YCrCb, or L in Lab) contribute to image understanding, and what insights can they offer that RGB cannot?

REFERENCES:

Image Processing Lab (N-PECCS502P)

- https://docs.opencv.org/3.4/de/d25/imgproc_color_conversions.html
- <https://www.tutorialspoint.com/color-spaces-in-opencv-and-python>
- <https://learnopencv.com/color-spaces-in-opencv-cpp-python/>
- <https://www.geeksforgeeks.org/python/python-visualizing-image-in-different-color-spaces/>
- <https://www.dynamsoft.com/blog/insights/image-processing/image-processing-101-color-models/>